

VisualLesson AI

A Quality-Gated Lesson Studio for High-School Learning

Product White Paper

Meiyu Shen

Yifan Yan

Jiachen Yu

Jiaxing Senior High School (BC Offshore Program)

August 2026

Abstract

We present VisualLesson AI, a public web application that turns a high-school question into a structured explanation, source-grounded notes, a quiz, a connected concept map, and an optional narrated visual lesson. We designed the system around staged generation rather than a single unrestricted model call. A text gate checks structure and then applies a compact five-dimension semantic review. A visual gate checks frame health in parallel and uses at most one semantic contact-sheet review. The application accepts user-owned API keys, supports multiple text and image providers, and keeps those keys only in the active Streamlit session. We create video by composing seven generated images with captions and narration; we do not use direct text-to-video generation because its cost is difficult to justify for the intended classroom workflow. We also provide full and visual-only regeneration after a lesson is complete. A connected-topic view links the current lesson to prerequisites, applications, contrasts, and next steps, and every related topic receives deterministic links to trusted learning platforms. This white paper describes the current product, the choices behind it, the evidence we retain from checker experiments, and the limits that remain. The live application is available at <https://teachhighschool.streamlit.app/>, and the implementation is available at <https://github.com/allanzhang721/AiLessonStudio>.

1 Introduction

Generative models can create educational material quickly, but speed is not the same as a useful lesson. A fluent answer can omit a condition. A polished image can teach the wrong relationship. A video can look complete while its frames are unreadable. These failures are especially important for high-school learners because students may not yet have enough background knowledge to detect them.

We developed VisualLesson AI around this practical problem. A student should be able to ask a focused question, select a grade and subject, connect a preferred model provider, and receive a lesson that is easier to inspect than a single long response. We therefore separate explanation, evidence, practice, visual media, and quality reporting. We also expose the path taken by each quality gate, including its score, latency, and model-call count.

The current system is different from the original research prototype. The prototype treated fine-tuned text and image classifiers as the production gates. Our experiments remain useful for comparing methods, but the saved checkpoints do not answer every question the deployed application needs to answer. The text classifiers predict pedagogical error categories; they do not establish whether a newly generated explanation is factually correct. The image-text probes measure alignment on a research dataset; they do not directly measure caption readability, exposure, rendering quality, or continuity in a newly generated lesson. We therefore keep the trained models as offline baselines and use a lightweight local-first cascade in production.

We make five product contributions. First, we provide a provider-flexible application in which users supply their own text and image API keys. Second, we generate layered lessons with explanations, worked examples, common mistakes, quizzes, sources, and follow-up topics. Third, we use two cost-aware quality gates that stop downstream spending when an earlier artifact fails. Fourth, we build narrated video from still images rather than calling an expensive direct video model. Finally, we connect each lesson to a small sourced concept neighborhood and allow the user to regenerate either the complete lesson or only its visual media.

2 Product Goal and Boundary

We target short explanatory lessons for students in Grades 9–12. The system is useful for self-study, teacher preparation, and rapid exploration of a difficult concept. It is not a replacement for a teacher, a textbook, or a formal assessment system. We treat automated quality checks as risk reduction rather than proof of correctness.

The product follows four design principles.

1. **Inspect before spending.** We check the explanation before web research, image generation, narration, and video composition.
2. **Use the cheapest useful method first.** Local checks run before semantic model calls, and technical frame checks run before vision review.
3. **Separate evidence from generation.** The lesson model does not invent citations. A separate web-grounded step produces source notes and clickable references.
4. **Keep the learner in control.** The interface exposes providers, models, cost-sensitive options, quality reports, regeneration controls, and source links.

These principles define the product boundary. We do not claim that a passing gate makes a lesson error-free. We do not train on a student’s private API traffic. We do not store visitor keys in files, logs, or environment variables. We also do not generate video directly from text. The current media path is intentionally explicit: plan, images, captions, narration, and composition.

3 Current System

3.1 Input and provider selection

A lesson begins with a question q , subject s , grade g , and language ℓ . The user also chooses a text provider and model. OpenAI and DeepSeek are available for lesson text. OpenAI and Alibaba Wan are available for images. Narration uses an OpenAI speech model. The application asks for the required keys in the sidebar and stores them only in Streamlit session state.

There is no demo mode. We made this choice because a demo response can hide provider failures and make it unclear whether a lesson was actually generated. The application instead shows whether the current key configuration is ready. Text-only use does not require an image key. Visual lesson generation requires an image key and a narration key, and the interface warns that seven image calls can have noticeable cost.

3.2 End-to-end flow

Let the user request be

$$x = (q, s, g, \ell).$$

The product returns a lesson bundle

$$y = (e, r, z, m, F, v),$$

where e is the approved explanation, r is the cited research report, z is the quiz, m is the connected concept map, $F = (I_1, \dots, I_7)$ is the ordered frame set, and v is the optional narrated MP4.

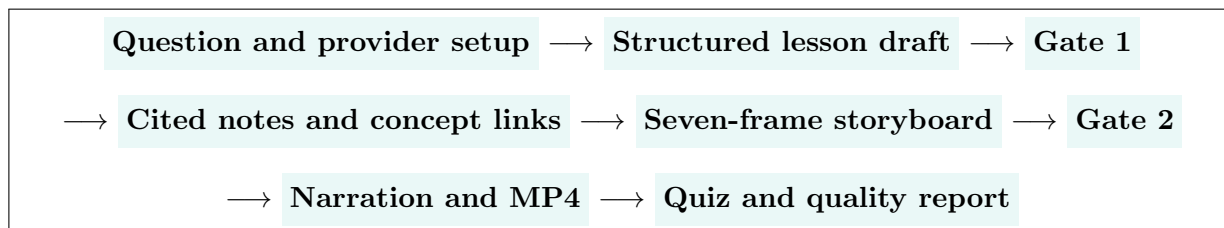


Figure 1: The current product flow. Expensive media stages begin only after Gate 1 passes. Gate 2 is persisted immediately after frame review, even if narration or MP4 assembly later fails.

The application records stage times and stores the checker output in the run manifest. It also distinguishes a gate that failed from one that did not apply. Gate 2 can therefore be *waiting*, *blocked*, *not completed*, *not applicable*, *passed*, or *needs review*. This distinction matters. A text-only lesson should not look like a failed visual check, and a narration error should not erase a visual check that already completed.

4 The Lesson Experience

4.1 Layered explanation

The text model returns a strict JSON lesson bundle. We normalize that bundle before rendering it. The student sees a clear explanation, key ideas, prerequisites, a worked example, a retrieval question, common traps, useful connections, and a short study path. We use tabs to keep these layers visible without placing everything in one long page.

The explanation and worked example preserve Markdown paragraphs and lists. We also normalize common mathematical wrappers. Inline expressions written with parenthesized LaTeX delimiters become dollar-delimited math, while bracketed delimiters, equation environments, and math code fences become display math. This keeps model-authored formulas compatible with Streamlit’s KaTeX rendering.

4.2 Practice and feedback

Each lesson contains five multiple-choice questions. After submission, the application shows the correct answer and a short explanation. It also summarizes the concepts that the student missed. This is a lightweight review signal, not a longitudinal mastery estimate. We prefer a transparent list of concepts to an opaque score that would require more interaction data than one lesson can provide.

4.3 Evidence and sources

When cited research is enabled, we use a separate grounded search call after Gate 1 passes. The report is divided into what reliable sources agree on, notes from each source, and evidence limits. URL annotations are converted into visible Markdown links. We prioritize government agencies, universities, museums, standards bodies, peer-reviewed work, and established open textbooks.

We keep this evidence path separate from the initial lesson draft. The generator is instructed not to invent books, URLs, statistics, or named studies. A citation shows where a claim came from; it does not guarantee that every source is correct or that the lesson covers every viewpoint. The interface states this limitation directly.

5 Quality Gate 1: Explanation Review

Gate 1 is a local-then-semantic cascade. The local screen checks length, focus, overlap with key question terms, and sentence density. These checks are inexpensive and typically complete in milliseconds. They identify structural problems, but they do not claim to verify facts.

A compact semantic review then scores five dimensions from 1 to 4:

$$D_1 = \{\text{accuracy, completeness, logical flow, grade fit, clarity}\}.$$

The explanation passes only when accuracy is 4 and every other dimension is at least 3. If the first review fails and the model returns a correction, we permit one repair and one recheck. Thus the normal path uses one model call, while the repair path uses two. We cap the loop because repeated self-revision can increase cost without providing independent evidence.

This design replaces the original production claim that a DistilBERT error classifier decides whether the explanation is safe. We still report the classifier experiments, but we use them for what they measure. DistilBERT, BERT, and RoBERTa classify error types on the saved dataset. Their performance does not establish factual correctness for an arbitrary new lesson.

The gate returns its final explanation, normalized score, issues, rounds, local and semantic latency, model-call count, repair count, and method comparison. In practice, this record makes the decision easier to inspect and makes cost behavior easier to understand.

6 Quality Gate 2: Visual Review

Gate 2 begins only after all seven frames are available. We calculate technical metrics for multiple frames in parallel. The metrics cover resolution, exposure, contrast, edge detail, and variation in the caption band. Each frame receives a score and a list of concrete issues such as low resolution, poor exposure, low contrast, low detail, or weak caption-band readability.

The gate uses a local-first decision rule. Let $t_i \in [0, 1]$ be the technical score of frame i . If the average technical score is below its threshold, or any required frame fails, we stop before semantic review. This short circuit makes the failure fast and uses no vision-model call. If the frames pass and a compatible vision client is available, we combine them into one compressed contact sheet and make one semantic call. The reviewer scores semantic accuracy, teaching alignment, sequence continuity, legibility, and visual load.

For a completed semantic review, the displayed score is

$$S_2 = 0.35S_{\text{technical}} + 0.65S_{\text{semantic}}.$$

The score is useful for reporting, but the pass decision also requires both component checks to pass. A high semantic score therefore cannot hide an unreadable frame, and strong pixel statistics cannot establish that a diagram teaches the right concept.

The original trained backend is not loaded in the public app. If it is requested without a production model integration, the gate falls back to the parallel heuristic path instead of breaking lesson generation. This is deliberate. The saved CLIP probes are research artifacts, and the public application avoids heavyweight local ML dependencies to keep startup light.

7 Visual Lesson and Video Pipeline

The planner turns the approved explanation into seven ordered teaching steps. Each step specifies a goal, what must remain, what must be added, and what must be avoided. The image pipeline creates

the frames from this shared plan. Captions use a reserved lower band so the visual explanation and the narration follow the same order.

We then synthesize a clean teaching voice and compose the still frames, captions, transitions, and audio into an H.264 MP4. Scene durations are derived from narration length and constrained by a minimum reading time. This produces a predictable artifact that can be inspected frame by frame.

We removed the direct text-to-video path. Direct video generation is expensive, difficult to make deterministic, and harder to validate at the level of individual teaching steps. The image-composition path gives us seven explicit checkpoints, stable captions, controllable timing, and a recoverable intermediate artifact. It is not as cinematic as a direct video model, but it is better aligned with the instructional and cost constraints of this product.

Gate 2 is saved immediately after frame validation. This sequencing fixes an important reliability problem in the earlier interface. Previously, the application stored the entire pipeline result only after narration and MP4 assembly. If audio failed, the completed visual audit disappeared and the interface incorrectly showed that Gate 2 had not run. We now publish the gate result before later media stages, so the visible state matches the work that actually completed.

8 Regeneration

Generation is probabilistic, and a user may prefer a different explanation or visual style even when both gates pass. We therefore expose two post-generation actions.

- **Regenerate full lesson** creates a new explanation and repeats the enabled research and media stages using the saved question, subject, grade, and language.
- **Regenerate visuals only** keeps the approved explanation but creates a fresh storyboard, seven images, narration, and MP4.

The interface states the cost difference before the user acts. Visual regeneration makes seven new image calls. We retain the existing lesson until the replacement path produces new state, and we use the same immediate Gate 2 persistence described above. This lets a student retry presentation without losing the conceptual content that already passed Gate 1.

9 Connected Concept Map

A useful lesson should show what comes next. We ask the text model for five short related topics and a typed relationship for each topic: prerequisite, application, contrast, or next step. The model also explains why each connection is useful. We discard any URL returned by the model.

The application renders these nodes as a directed graph centered on the current lesson. A student can click a node to open its first source. Below the graph, each concept receives a study card with two links. One link searches Khan Academy using the current subject and topic. The second comes from a deterministic subject mapping: OpenStax for physics, biology, and chemistry; GeoGebra for mathematics; MDN for computing; and OpenStax’s subject library for a general topic.

This view is inspired by connected-literature tools, but its purpose is different. We are not claiming citation similarity between papers. We represent a small learning neighborhood around one lesson. The edges describe instructional relationships, and the links lead to study resources. Because the source domains are fixed in code, the language model proposes concepts but does not control where students are sent.

10 Offline Checker Evidence

The repository contains saved experiments for trained and untrained checker variants. Table 1 summarizes the results shown in the application. We call these offline experiment metrics. They are not live request latency, and the Gate 1 and Gate 2 datasets are different.

Gate	Method	Accuracy	Precision	Recall	F1	AUROC
1	DistilBERT classifier	0.9280	0.9444	0.9280	0.9284	–
1	BERT classifier	0.9253	0.9314	0.9253	0.9249	–
1	RoBERTa classifier	0.9653	0.9667	0.9653	0.9655	–
2	CLIP cosine threshold	0.9628	0.9474	0.9800	0.9634	0.9922
2	CLIP linear probe	0.8756	0.8301	0.9444	0.8836	0.9555
2	CLIP MLP probe	0.9517	0.9462	0.9578	0.9520	0.9885

Table 1: Saved checker experiments. Gate 1 methods classify pedagogical error categories. Gate 2 methods classify aligned and misaligned Flickr8k pairs. These values support method comparison but do not directly estimate production lesson correctness.

The best Gate 1 result in this comparison is RoBERTa. The best Gate 2 result is the untrained CLIP cosine threshold, which also exceeds the trained probes in this experiment. This result is one reason we do not assume that a trained head is automatically the better deployment choice. The operational decision also considers model size, startup time, the match between the benchmark task and the product failure, and whether the output is interpretable.

We surface both kinds of evidence in the application. The home page shows the fixed offline benchmark snapshot. After generation, the quality report shows live gate latency, decision path, score, model-call count, and per-method details for the current lesson. Keeping these views separate prevents experiment runtime from being mistaken for per-request latency.

11 Implementation, Privacy, and Operations

The public interface uses Streamlit and Python 3.12. The production dependency set remains small: the provider SDK, Pillow, ImageIO, FFmpeg integration, Streamlit, Requests, and Graphviz. Training libraries and research datasets are separated from the deployment requirements. This reduces cold-start cost and avoids loading checkpoints that the product does not use.

Visitor keys are passed directly to provider clients and are not copied into environment variables. They stay in the current session and are not written to lesson JSON, manifests, logs, or repository files. A deployment owner may still configure an optional server-side OpenAI secret, but the normal interface asks each visitor to provide the required keys.

The application is deployed from the `agent/streamlit-production` branch, while completed changes are also merged into `main`. The current quality and interface work is covered by 30 deployable production tests. These tests cover provider selection, schema validation, fail-fast rendering, Markdown and math normalization, frame scoring, semantic short-circuiting, trained-backend fallback, Gate 2 persistence, video composition, source annotation conversion, and concept-map URL safety.

12 Limitations

The product has five important limitations.

First, the semantic gates can share weaknesses with the generator, especially when both use related model families. A passing result is not independent expert review. Second, technical frame metrics are proxies. Edge variation can detect low detail, but it cannot determine whether every arrow or label is scientifically correct. Third, the trusted concept links lead to reliable platforms, but some are search or textbook entry points rather than exact passages selected for the generated claim. Fourth, image and narration costs remain external and vary by provider. Regeneration makes those calls again. Fifth, Streamlit session state is temporary. A browser refresh or session expiration can remove generated state unless the user downloads the lesson or video.

The saved experiments also have limits. The Checker 1 dataset measures error-type classification, and the Checker 2 dataset uses aligned and mismatched Flickr8k pairs. Neither dataset is a full evaluation of learning outcomes. We do not claim that the benchmark numbers establish classroom effectiveness.

13 Roadmap

We see four useful next steps. We can map web-grounded citations directly to individual concept nodes instead of relying on curated platform searches. We can add teacher approval and editing before media generation, which would create a stronger human checkpoint. We can persist lesson history without storing API keys and allow a student to compare regenerated versions. Finally, we can evaluate the complete lesson with teachers and students, measuring factual corrections, time saved, comprehension, and which visual failures the current gate misses.

14 Conclusion

We built VisualLesson AI as a staged lesson studio rather than a one-call content generator. The system separates explanation, evidence, visuals, assessment, and connected study. Gate 1 checks the explanation before expensive work. Gate 2 checks seven frames locally and uses at most one semantic contact-sheet call. The video path composes images and narration instead of using direct video generation. Regeneration gives the user control after completion, and the concept map turns one answer into a sourced path toward related topics.

The main design lesson is simple. More model complexity does not automatically produce a better educational product. We use trained experiments to understand alternatives, local checks to catch cheap failures, semantic review where meaning matters, and visible states so the student can tell what happened. This combination keeps the application light enough to deploy while making its decisions easier to inspect.

Document Note

This white paper reflects the application state in August 2026. Generative AI was used to assist with organization, grammar, and readability. The project implementation, experiment records, and final claims were reviewed against the repository state. The uploaded SLaTR manuscript was used only as a reference for writing habits and sentence structure; no technical wording or claims were copied from it.

References

- [1] R. E. Mayer. *Multimedia Learning*. Cambridge University Press, second edition, 2009.

- [2] E. Kasneci et al. “ChatGPT for good? On opportunities and challenges of large language models for education.” *Learning and Individual Differences*, 103:102274, 2023.
- [3] V. Sanh et al. “DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter.” arXiv:1910.01108, 2019.
- [4] J. Devlin et al. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.” NAACL-HLT, 2019.
- [5] Y. Liu et al. “RoBERTa: A Robustly Optimized BERT Pretraining Approach.” arXiv:1907.11692, 2019.
- [6] A. Radford et al. “Learning Transferable Visual Models From Natural Language Supervision.” ICML, 2021.
- [7] M. Hodosh, P. Young, and J. Hockenmaier. “Framing Image Description as a Ranking Task.” *Journal of Artificial Intelligence Research*, 47:853–899, 2013.